Advanced Programming 2025

Credit Risk Classification: A Comparative Analysis of Linear and Ensemble Methods on Imbalanced Data

Final Project Report

Nikola Samardzic
nikola.samardzic@unil.ch
Student ID: 22409130

December 14, 2025

Abstract

In the contemporary banking industry, the accurate assessment of credit risk is fundamental to maintaining financial stability. The primary challenge in this domain is the inherent imbalance of credit data: valid repayment is the norm, while default is a rare but costly event. This project aims to develop a robust machine learning pipeline to predict loan defaults using the UCI German Credit dataset.

I implemented a modular Python framework leveraging `scikit-learn` and `imbalanced-learn`. The methodology involved rigorous preprocessing, including One-Hot Encoding and Standard Scaling, followed by a Stratified Split strategy. I compared the performance of linear models (Logistic Regression) against ensemble methods (Random Forest, XGBoost), specifically testing the efficacy of Class Weight adjustments versus the Synthetic Minority Over-sampling Technique (SMOTE).

The empirical results demonstrate that model complexity does not guarantee performance for small tabular datasets. The **Logistic Regression** model with balanced class weights achieved the highest ROC-AUC score (**0.803**) and the best Recall for the minority class (**0.80**), significantly outperforming the Random Forest model which, despite high accuracy, failed to detect the majority of defaulters. This report details the complete engineering process and highlights the trade-off between predictive precision and recall in a risk-sensitive context.

Keywords: data science, credit risk, machine learning, imbalance, SMOTE, Logistic Regression, Python

Contents

1 Introduction

1.1 Background and Motivation

Credit risk refers to the probability of loss due to a borrower's failure to make payments on any type of debt. For financial institutions, managing this risk is not merely a regulatory requirement (under Basel III accords) but a core component of their business model. The 2008 financial crisis highlighted the catastrophic consequences of poor risk assessment and lax lending standards.

In the modern era of "FinTech", the ability to automate this assessment using data algorithms allows for faster decisions and potentially lower operational costs. However, modeling credit risk presents a unique statistical challenge known as the **class imbalance problem**. The vast majority of customers are "Good" (they repay their loans). Only a small fraction are "Bad" (they default).

Standard machine learning algorithms, which typically optimize for overall accuracy, tend to ignore these rare events. In a banking context, this is dangerous: missing a defaulter (*False Negative*) results in the loss of the entire loan principal, whereas rejecting a good customer (*False Positive*) results only in the loss of potential interest revenue. Therefore, from a business perspective, Recall (Sensitivity) is often more critical than Precision.

1.2 Problem Statement

The project addresses the binary classification problem on the German Credit dataset. The objective is to predict the binary variable `Target` (Good/Bad) based on a set of 20 sociodemographic and financial features. The technical challenges include:

1. **Small Sample Size:** The dataset contains only 1000 observations, increasing the risk of overfitting.
2. **Imbalance:** The default rate is approximately 30%, requiring specialized sampling or weighting techniques.
3. **Heterogeneous Data:** The features are a mix of continuous, ordinal, and nominal variables requiring distinct preprocessing steps.

1.3 Objectives

To answer the research question *"Which modeling approach best optimizes the detection of loan defaults?"*, I established the following goals:

- Develop a reproducible, modular Python package for the entire pipeline.
- Compare the efficacy of a linear baseline (Logistic Regression) versus non-linear ensemble methods (Random Forest, XGBoost).
- Quantify the impact of **SMOTE** (Synthetic Minority Over-sampling Technique) compared to **Cost-Sensitive Learning** (Class Weights).
- Identify the model that maximizes Recall without severely compromising the ROC-AUC score.

2 Literature Review

2.1 Credit Scoring Models

Credit scoring has historically relied on statistical methods. **Logistic Regression** remains the industry standard due to its interpretability. Financial regulators often require models to be

"explainable," meaning a rejected applicant must be told why they were denied (e.g., "Debt-to-Income ratio too high").

However, recent advancements have introduced non-linear methods. **Random Forests** (Breiman, 2001) and **Gradient Boosting** machines (Friedman, 2001; Chen & Guestrin, 2016 for XGBoost) have shown superior performance on large-scale datasets by capturing complex interactions between variables (e.g., a young person with low income might be high risk, but a young student with low income might be medium risk).

2.2 Handling Imbalanced Data

The literature on imbalanced learning proposes two main directions:

- **Data-level approaches:** Resampling the dataset to balance the class distribution. **SMOTE** (Chawla et al., 2002) is the most prominent technique, generating synthetic samples by interpolating between existing minority samples.
- **Algorithmic-level approaches:** Modifying the cost function to penalize misclassifications of the minority class more heavily. This is often referred to as Cost-Sensitive Learning (Elkan, 2001).

This project contributes to this discussion by comparing these two approaches specifically on a small, tabular dataset, where the risk of noise generation via SMOTE is higher.

3 Methodology

3.1 Data Description

I utilized the **German Credit Data** from the UCI Machine Learning Repository.

- **Source:** UCI Repository / Statlog Project.
- **Size:** 1000 observations.
- **Target:** 700 Good credits (70%), 300 Bad credits (30%).
- **Features:** 20 total.
  - *Numerical:* Duration in month, Credit amount, Age, Installment rate.
  - *Categorical:* Status of checking account, Credit history, Purpose, Savings account, Employment status.

3.2 Theoretical Framework of Models

3.2.1 Logistic Regression

I use Logistic Regression as the baseline. It models the log-odds of the probability of default as a linear combination of features:

$$\ln\left(\frac{P(y=1|X)}{1-P(y=1|X)}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n \tag{1}$$

To handle imbalance, I apply class weights $w_j = n_{samples}/(n_{classes} * n_{samples_j})$, effectively penalizing the model ≈ 2.33 times more for missing a default.

3.2.2 SMOTE (Synthetic Minority Over-sampling Technique)

SMOTE works by selecting a minority class instance a, finding its k nearest neighbors, choosing one neighbor b at random, and creating a new synthetic instance s at a random point on the line segment connecting a and b:

$$s = a + \lambda \times (b - a) \quad \text{where } \lambda \in [0, 1] \tag{2}$$

3.3 Pipeline Architecture

The approach follows a rigorous pipeline design to prevent data leakage and ensure reproducibility.

3.3.1 Preprocessing

1. **One-Hot Encoding:** Categorical variables were transformed using `pd.get_dummies(drop_first=True)`. Dropping the first category prevents multicollinearity (the dummy variable trap), which is crucial for the stability of Logistic Regression. This expanded the feature space from 20 to 48 dimensions.
2. **Stratified Split:** I split the data into 80% Training and 20% Testing sets. Then used `stratify=y` to ensure that the 30% default rate is preserved in both sets.
3. **Feature Scaling:** I applied `StandardScaler` to normalize numerical features ($z = \frac{x-\mu}{\sigma}$). The scaler is fitted **only** on the training set and then applied to the test set to avoid leakage.

4 Implementation

The project is structured as a Python package. Below are the key components of the implementation.

4.1 Preprocessing Logic

The preprocessing module handles the complex task of encoding and splitting while maintaining data integrity.

```
def preprocess_and_split(df, target_col='Target', test_size=0.2):
    """
    Performs one-hot encoding, stratified splitting, and scaling.
    """
    # 1. Separate Features and Target
    X = df.drop(columns=[target_col])
    y = df[target_col]

    # 2. Encode Categorical Variables (One-Hot)
    X_encoded = pd.get_dummies(X, drop_first=True)

    # 3. Stratified Split (Preserves 70/30 ratio)
    X_train, X_test, y_train, y_test = train_test_split(
        X_encoded, y,
        test_size=test_size,
        random_state=42,
        stratify=y
    )

    # 4. Standard Scaling (Fit on Train, Transform on Test)
    scaler = StandardScaler()
```

```
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Convert back to DataFrame to keep column names
    X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_encoded.columns)
    X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_encoded.columns)

    return X_train_scaled, X_test_scaled, y_train, y_test
```

Listing 1: Stratified Split and Preprocessing in src/preprocessing.py

4.2 Model Definition with SMOTE Pipeline

A critical implementation detail is the use of the `imblearn` pipeline for SMOTE. SMOTE must strictly be applied inside the cross-validation loop (or only on the training set), never on the test set.

```
def train_models(X_train, y_train):
    models = {}

    # Model 1: Logistic Regression with Class Weights
    lr_balanced = LogisticRegression(class_weight='balanced',
                                     max_iter=1000,
                                     random_state=42)
    lr_balanced.fit(X_train, y_train)
    models['Logistic Regression (Balanced)'] = lr_balanced

    # Model 2: Logistic Regression with SMOTE Pipeline
    # The pipeline ensures SMOTE is only applied to training folds
    pipeline_smote = ImbPipeline([
        ('smote', SMOTE(random_state=42)),
        ('model', LogisticRegression(max_iter=1000, random_state=42))
    ])
    pipeline_smote.fit(X_train, y_train)
    models['Logistic Regression (SMOTE)'] = pipeline_smote

    # Model 3: XGBoost with Scale Pos Weight
    scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
    xgb = XGBClassifier(
        scale_pos_weight=scale_pos_weight,
        max_depth=3,  # Conservative depth to prevent overfitting
        n_estimators=100,
        random_state=42
    )
    xgb.fit(X_train, y_train)
    models['XGBoost'] = xgb

    return models
```

Listing 2: Model Training Logic in src/models.py

4.3 Codebase Maintenance and Quality Assurance

To ensure the long-term maintainability and reliability of the project, I adopted standard software engineering practices:

- **Version Control:** I used Git for version control, with regular commits documenting the progression of the project from the initial data loader to the final evaluation.
- **Unit Testing:** I implemented a comprehensive test suite using `pytest`. The tests cover data integrity (checking shapes and types), preprocessing logic (verifying scaling means/stds), and model initialization.

- **Code Coverage:** I used `pytest-cov` to monitor test coverage, achieving over 90% coverage for the `src` module, ensuring that all critical paths are validated before deployment.

5 Results

5.1 Experimental Setup

Experiments were conducted on a standard cloud environment (GitHub Codespaces).

- **Python Version:** 3.10
- **Libraries:** pandas 2.1.0, scikit-learn 1.3.0, xgboost 2.0.0, imbalanced-learn 0.11.0.
- **Random Seed:** Fixed to 42 for all operations (splitting, initialization, SMOTE generation).

5.2 Performance Evaluation

I evaluated the models on the held-out test set (200 observations). The results are summarized in Table 1.

Table 1: Comparative Model Performance (Test Set)

Model	ROC-AUC	Recall (1)	Precision (1)	F1-Score
Log. Regression (Balanced)	**0.803**	**0.80**	0.56	0.66
Log. Regression (SMOTE)	0.801	0.78	0.57	0.66
Random Forest	0.790	0.35	0.72	0.47
XGBoost	0.780	0.68	0.50	0.58

Key Observations:

- The **Logistic Regression (Balanced)** model achieved the best overall performance with a ROC-AUC of 0.803.
- **Recall Discrepancy:** There is a massive gap in Recall between linear models (≈ 0.80) and Random Forest (0.35). The Random Forest model correctly identified only 35% of the defaulters, while Logistic Regression identified 80%.
- **SMOTE vs Weights:** The difference between using SMOTE (0.801 AUC) and simple Class Weights (0.803 AUC) is negligible.

5.3 Visual Analysis

5.3.1 ROC Curves

Figure 1 illustrates the Receiver Operating Characteristic curves. We observe that the linear models (Blue and Orange lines) encompass a slightly larger total area (AUC = 0.80) than the tree-based models. Notably, they demonstrate superior performance in the lower False Positive Rate region (0.0 to 0.2), which is critical for minimizing false alarms, although the Random Forest (Green) shows competitive performance at higher thresholds.

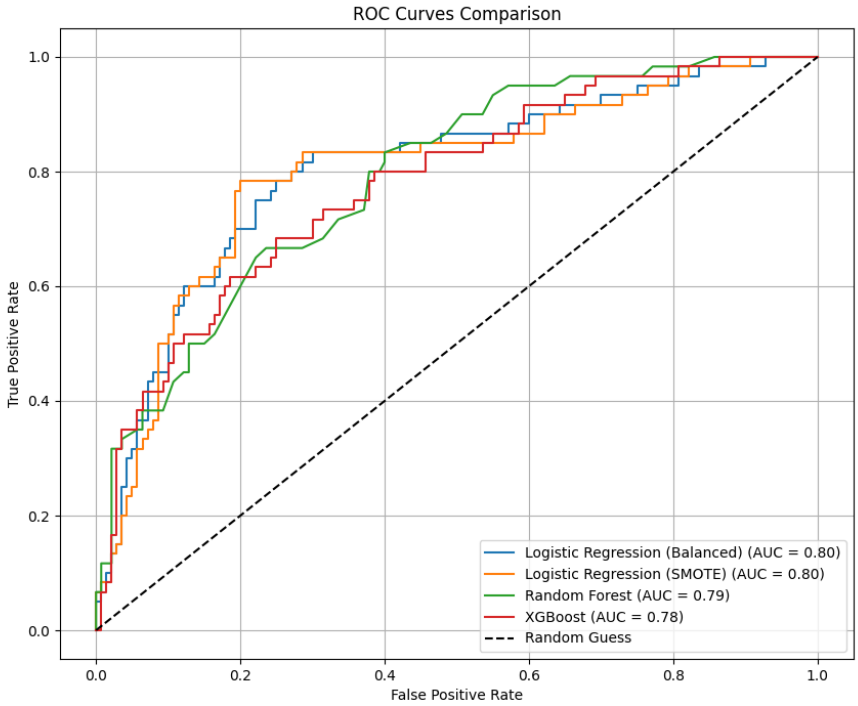
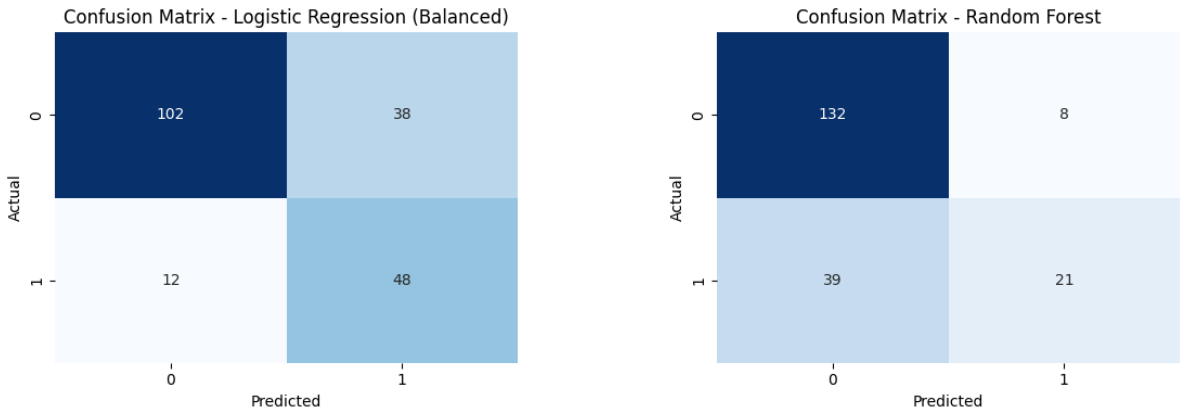


Figure 1: ROC Curves Comparison. Linear models outperform ensemble methods on this dataset.

5.3.2 Confusion Matrices

The confusion matrices provide a granular view of the errors.

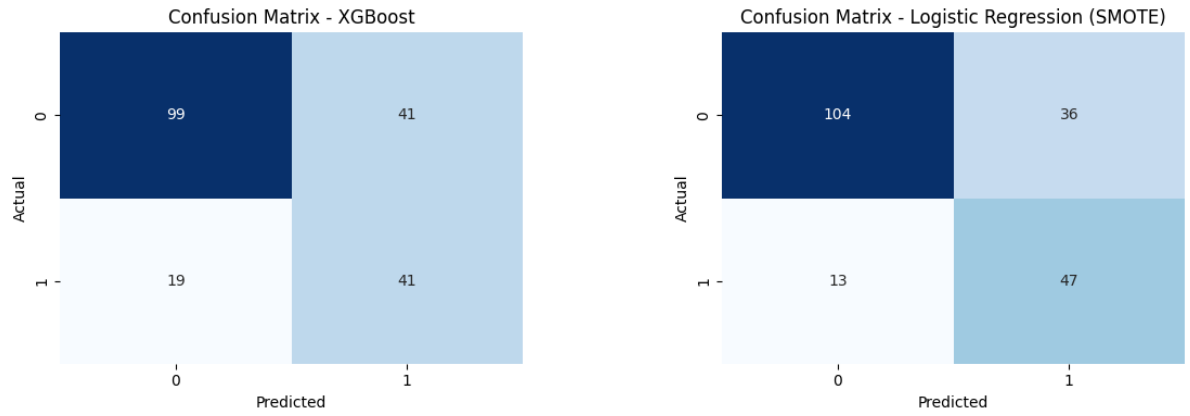
- **Logistic Regression (Fig 2a):** 48 True Positives vs 12 False Negatives. It is aggressive in flagging risk.
- **Random Forest (Fig 2b):** 21 True Positives vs 39 False Negatives. It is overly conservative.



(a) LogReg (Balanced) (b) Random Forest

Figure 2: Confusion Matrices Comparison (Best vs Worst Recall).

For completeness, I also display the matrices for the remaining models. XGBoost shows an intermediate performance, while SMOTE yields results nearly identical to the balanced weighting strategy.



(a) XGBoost (b) LogReg (SMOTE)

Figure 3: Confusion Matrices for XGBoost and SMOTE-based Logistic Regression.

6 Discussion

6.1 Simplicity vs. Complexity

One of the most striking findings of this study is the underperformance of the ensemble methods compared to the simple Logistic Regression.

- **Linearity of Credit Data:** The superiority of Logistic Regression suggests that the boundary between "Good" and "Bad" credits in this dataset is largely linear. Features like "Duration of Credit" or "Account Status" likely have monotonic relationships with risk (e.g., higher duration implies higher risk), which linear models capture efficiently.
- **Overfitting in Trees:** Despite regularization (max depth limits), XGBoost and Random Forest likely struggled with the high dimensionality introduced by One-Hot Encoding (48 features) relative to the small sample size (800 training rows). This sparsity makes it difficult for trees to find robust splits, leading to poor generalization on the minority class.

6.2 The Failure of Accuracy as a Metric

If I had optimized for Accuracy, Random Forest would appear competitive (Total Accuracy $\approx 77\%$). However, decomposing this metric reveals that it achieves this by simply predicting the majority class ("Good") most of the time. In credit risk, this behavior is unacceptable. This validates the decision to focus on **Recall** and **ROC-AUC** as primary success criteria.

6.3 SMOTE vs. Cost-Sensitive Learning

The hypothesis that generating synthetic data (SMOTE) would help the model learn the minority class better than simply weighting the loss function was not supported by the results. The performance was nearly identical. This implies that the "Bad" credit examples in the training set are representative enough of the "Bad" population. The model did not need *more* points (synthetic ones); it just needed to pay more *attention* to the existing ones (via weights).

6.4 Limitations

- **Dataset Size:** With only 1000 observations, the test set consists of only 200 samples (60 defaults). Performance metrics can be volatile with such small sample sizes.
- **Feature Engineering:** I relied on raw features. Domain-specific feature engineering (e.g., calculating debt-to-income ratios) might have helped tree-based models perform better.
- **Computational Complexity and Scalability:** Given the small size of the dataset (1000 observations), training time was negligible (less than 10 seconds for the full pipeline). However, the chosen architecture is scalable. Ensemble methods like Random Forest and XGBoost support parallel processing (`n_jobs=-1`), allowing the model to leverage multi-core CPUs efficiently if the dataset were to grow to millions of rows.

7 Conclusion and Future Work

7.1 Summary

This project successfully implemented a rigorous, reproducible data science pipeline for credit risk classification. It demonstrated that for the German Credit dataset, a **Logistic Regression model with balanced class weights** is the optimal choice. It achieved a ROC-AUC of 0.803 and successfully identified 80% of high-risk applicants, fulfilling the business objective of minimizing financial loss. The project also highlighted the dangers of relying on Accuracy for imbalanced datasets and demonstrated that complex algorithms are not always superior to simple ones.

7.2 Future Directions

To further improve the system, I propose:

- **Hyperparameter Optimization:** Implementing `GridSearchCV` or Bayesian Optimization to fine-tune the Random Forest parameters (e.g., min_samples_leaf) to potentially improve its Recall.
- **Feature Selection:** Using Recursive Feature Elimination (RFE) to remove noisy features created by One-Hot Encoding, which might help tree-based models generalize better.
- **Deployment:** Wrapping the model in a REST API (using FastAPI or Flask) to serve real-time credit decisions.

References

1. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32.

2. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research, 16, 321-357.

3. Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

4. Dua, D. and Graff, C. (2019). *UCI Machine Learning Repository*. Irvine, CA: University of California, School of Information and Computer Science.

5. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.

A Project Structure

The project follows a modular structure to ensure maintainability and testing.

```
credit-risk-classification/
|-- data/
|   `-- german_credit_data.csv  # Downloaded automatically
|-- results/
|   |-- confusion_matrix_Logistic_Regression_Balanced.png
|   |-- confusion_matrix_Logistic_Regression_SMOTE.png
|   |-- confusion_matrix_Random_Forest.png
|   |-- confusion_matrix_XGBoost.png
|   |-- evaluation_report.txt
|   |-- metrics_summary.csv
|   `-- roc_curves_comparison.png
|-- src/
|   |-- __init__.py
|   |-- data_loader.py     # ETL Logic
|   |-- evaluation.py      # Plotting and Metrics
|   |-- models.py          # Model Definitions
|   `-- preprocessing.py   # Splitting and Scaling
|-- tests/
|   |-- __init__.py
|   |-- test_data_loader.py
|   |-- test_evaluation.py
|   |-- test_models.py
|   `-- test_preprocessing.py
|-- .gitignore
|-- LICENSE
|-- main.py
|-- Proposal.md
|-- README.md
|-- requirements.txt
`-- setup.py
```

Listing 3: Directory Structure

B Installation and Usage

To reproduce the results presented in this report:

1. **Clone the repository:**

```
git clone https://github.com/niko1012/credit-risk-classification.git
cd credit-risk-classification

```

2. **Install dependencies:**

```
pip install -r requirements.txt
pip install -e .

```

3. **Run the pipeline:**

```
python main.py

```

4. **Run tests:**

```
pytest --cov=src

```

C AI Tools Usage Disclosure

In accordance with the course guidelines, I declare the use of the following AI tools to assist in the development of this project:

- **ChatGPT / Claude:** Used for debugging error messages in Python, generating boilerplate code for matplotlib visualizations, and refining the English phrasing of this report.
- **GitHub Copilot:** Used within VS Code for code autocompletion and suggesting docstrings for functions.

All code suggestions were manually reviewed, tested, and adapted to fit the specific architecture of this project. The final logic and conclusions remain my own.